

Structured Query Language (SQL)

DDL: Linguagem de Definição de Dados

Linguagem de Definição de Dados (DDL)

```sql
CREATE DATABASE loja_do_joao;

CREATE SCHEMA loja;

CREATE TABLE loja.cliente (
    id_cliente SERIAL PRIMARY KEY,
    nome VARCHAR(100) NOT NULL,
    cpf CHAR(11) UNIQUE NOT NULL,
    data_nascimento DATE,
    data_de_criacao TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Linguagem de Definição de Dados (DDL)

- **PRIMARY KEY (Chave Primária):**
  - Uma **PRIMARY KEY** é uma **restrição** que identifica de forma única cada linha em uma tabela de banco de dados. Não podem existir duas linhas com a mesma chave primária, e ela não pode conter valores NULL.

Sintaxe

```
CREATE TABLE Pessoa (
    PessoaID INT NOT NULL,
    Nome VARCHAR(100),
    PRIMARY KEY (PessoaID)
);
```

- **FOREIGN KEY (Chave Estrangeira):**
  - Uma **FOREIGN KEY** é uma chave usada para **vincular duas tabelas juntas**. É uma coluna ou uma combinação de colunas usadas para estabelecer e impor um link entre os dados de duas tabelas para controlar os dados que podem ser armazenados na tabela de foreign key.

Sintaxe

```
CREATE TABLE Pedido (
    PedidoID INT NOT NULL,
    PedidoNumero INT NOT NULL,
    PessoaID INT,
    PRIMARY KEY (PedidoID),
    FOREIGN KEY (PessoaID) REFERENCES Pessoa(PessoaID)
);
```

- **COMPOSITE KEY (Chave Composta):**
  - É uma **chave primária** que consiste em **<u>duas ou mais colunas usadas juntas</u>** para criar uma chave primária para uma tabela. Isso é usado quando você deseja ter uma chave primária que contenha múltiplos atributos.

Sintaxe

```
CREATE TABLE ProdutoPedido (
    PedidoID INT,
    ProdutoID INT,
    Quantidade INT,
    PRIMARY KEY (PedidoID, ProdutoID),
    FOREIGN KEY (PedidoID) REFERENCES Pedido(PedidoID),
    FOREIGN KEY (ProdutoID) REFERENCES Produto(ProdutoID)
);
```

Linguagem de Definição de Dados (DDL)

- O tipo de dado que você escolhe ao criar uma tabela em SQL pode depender do SGBD que você está usando.

- Cada SGBD pode suportar diferentes tipos de dados ou ter variações nos nomes e no comportamento dos tipos de dados.

Linguagem de Definição de Dados (DDL)

- **Exemplos**

  - **MySQL**: Utiliza tipos como VARCHAR, TEXT, INT, DATETIME, entre outros.

  - **PostgreSQL**: Também usa tipos como VARCHAR e TEXT, mas também possui tipos como **SERIAL** para chaves primárias **autoincrementáveis** e tipos específicos como **JSON** e **ARRAY**.

Linguagem de Definição de Dados (DDL)

- **Exemplos**

  - **SQLite**: Tende a ser mais flexível com os tipos de dados, seguindo um sistema de **tipagem dinâmica**, mas reconhece tipos comuns como **TEXT**, **INTEGER**, e **REAL**.

  - **Microsoft SQL Server**: Tem tipos como **NVARCHAR** para texto Unicode, **INT**, **DATETIME**, entre outros, e também tipos específicos para dados hierárquicos como **XML** e **HIERARCHYID**.

Atividade

1) Escolha um SGBD relacional de sua preferência que utilize SQL e:
    a) Realize uma pesquisa sobre os tipos de dados suportados pelo SGBD escolhido.
    b) Quais são os tipos de dados básicos disponíveis? (Por exemplo, tipos numéricos, de data e hora, strings, etc.)
    c) Existem tipos de dados exclusivos ou especializados oferecidos por esse SGBD? (Por exemplo, JSON em PostgreSQL)
    d) Qual é o tamanho máximo para tipos de string como VARCHAR?
    e) Como são tratados os dados binários?
    f) Apresente as vantagens e desvantagens.
        i) P/ cada tipo de dado importante, descreva possíveis vantagens e desvantagens de seu uso em contextos de desenvolvimento reais. Por exemplo: quando usar um tipo TEXT em vez de VARCHAR?